Sensiq

Technical Reports: CL-2026-42, July 2026

Simon Völkl, Johannes Schieder, Sebastian Rosner and Andre Tien Vu
Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

***Abstract*—**
***Index Terms*—**

I. Introduction

Reliable environmental monitoring is a recurring requirement in laboratories, production halls, and similar facilities, where ambient conditions directly affect experimental validity, product quality, and occupational safety. Yet many existing solutions force a trade-off between real-time visibility, long-term data availability, and operational cost, leaving operators without a unified view of their environment.

This report describes Sensiq, a microcontroller unit (MCU) based Internet of Things (IoT) architecture that closes this gap by combining ESP32 (Espressif Systems) edge devices with a serverless Amazon Web Services (AWS) cloud backend. Sensor readings are transmitted over Message Queuing Telemetry Transport (MQTT) secured with Transport Layer Security (TLS), validated and persisted in the cloud via AWS services for a live and historical view, and exposed through Representational State Transfer (REST) Application Programming Interface (API) endpoints. A React-based web frontend consumes these endpoints to visualize live and historical data, and threshold-driven email alerts notify responsible personnel of critical events. The remainder of this section presents the mission statement and the underlying motivation for the system.

A. Mission Statement

Sensiq is an end-to-end environmental monitoring system that enables laboratory operators, occupational safety officers, facility managers, and IT administrators to observe critical ambient parameters such as temperature, humidity, light intensity, gas concentration, and flame exposure in real time, while also providing scalable access to historical data for long-term analysis. Its overarching goal is to deliver reliable, low-latency live monitoring together with cost-efficient historical analytics on a fully serverless backend, without depending on vendor-locked third-party dashboards.

B. Motivation

In laboratory and small-scale production environments, deviations in temperature, humidity, gas concentration, or fire exposure can invalidate experiments, damage sensitive samples, or directly endanger personnel. Operators therefore need both an immediate view of current conditions and a complete record of past readings for quality assurance and incident analysis.

In practice, this combination is hard to obtain. Many low-cost setups remain confined to a single workstation and offer no programmatic access to their measurements, while turnkey commercial dashboards typically lock data into a vendor platform, charge per user, and restrict the business logic that can be layered on top. Historical data is frequently either discarded after a short retention window or stored in formats that do not scale to long observation periods, leaving gaps precisely where an audit trail would be needed.

Sensiq is motivated by this gap. By pairing an inexpensive ESP32 sensor node with a serverless cloud backend, the system aims to provide real-time visibility, durable historical storage, and programmatic access through open REST endpoints, while keeping infrastructure and operating cost low enough for routine use in academic and industrial laboratories. A detailed comparison with existing academic and commercial solutions follows in Section II.

C. Document Structure

The remainder of this report is organised as follows. Section II positions Sensiq against existing academic and commercial monitoring solutions and motivates the design decisions taken in this project. Section III introduces the conceptual building blocks of the system and presents the end-to-end architecture in a technology-agnostic manner.

The subsequent three sections refine this concept along the data flow: Section IV details the ESP32-based sensor node and its secure MQTT publication to AWS IoT Core; Section V describes the serverless cloud pipeline, covering both the low-latency live route and the long-term history route; and Section VI explains how end users consume the data through the React-based web frontend and how they are notified of critical events via email. Section VII assesses the resulting system with respect to testability and operating cost. Finally, Section VIII summarises the contribution of the report and outlines directions for future work.

II. Related Work

Position Sensiq against existing academic and commercial solutions. Discuss two reference works in depth (IoT lab-safety monitoring; performance of serverless IoT pipelines) and contrast them with the design decisions of this project. Close the section with a comparison table summarising key differentiators (real-time alerts, long-term audit capability, custom API/UI,

Please cite as: Simon Völkl, Johannes Schieder, Sebastian Rosner, and Andre Tien Vu. *Sensiq*. Tech. rep. CL-2026-42.
Ostbayerische Technische Hochschule Amberg-Weiden, Department of Electrical Engineering, Media, and Computer Science,
July 2026.

cost efficiency, laboratory focus). Template available at end of document: tab:comparison.

III. Technical Concept

High-level, technology-agnostic view of the system. This section introduces the conceptual building blocks and the end-to-end data flow. Detailed component-level descriptions follow in the Hardware, AWS Services, and User Interaction sections.

A. Technical Building Blocks

Enumerate the logical components of the system without binding them yet to concrete products: edge sensing node, secure transport channel, ingestion gateway, validation stage, hot-path store (live data), cold-path store (history), query API, notification channel, and user interface. Briefly justify the chosen programming languages (C++ firmware, Python lambdas, TypeScript infrastructure, React UI).

B. System Architecture

Present the architecture diagram (see template: fig:cloud-architecture) and walk the reader through the data flow: ESP32 -> MQTT/TLS -> AWS IoT Core -> validation Lambda -> dual persistence (DynamoDB for live, S3 + Athena for history) -> REST endpoints -> frontend / SNS.

IV. Hardware

Detailed description of the edge tier of the system.

A. ESP32 Sensor Node

Describe the ESP32 microcontroller, the selected sensors (DHT11 for temperature/humidity, KY-026 flame sensor, KY-028 thermistor, and any additional gas/light sensors), wiring, and on-device pre-processing. Include a photograph or schematic of the assembled node (see template: figure*).

B. MQTT Publication to AWS IoT Core

Explain the firmware-side MQTT client: topic structure, JSON payload schema , publish frequency, and the TLS / X.509 certificate-based authentication that secures the link between device and AWS IoT Core.

V. AWS Services

Detailed description of the cloud-side processing pipeline. Two functional routes are exposed by the backend: a low-latency live route and a long-term history route. Each is described in its own subsection together with the AWS services it relies on.

A. Live Route

Path from IoT Core rule -> validation Lambda -> DynamoDB (latest reading per device) -> API Gateway -> GET /live. Discuss latency targets, payload shape, and error handling.

B. History Route

Path from IoT Core / validation Lambda -> S3 (Parquet, partitioned by date) -> Glue catalogue -> Athena -> history Lambda -> API Gateway -> GET /history. Discuss partitioning strategy, query patterns, and expected query latency.

VI. User Interaction

Describes how end users consume the data and receive alerts.

A. Email Notifications via SNS

Explain the alert pipeline: threshold evaluation in the validation Lambda, publication to an SNS topic, and email delivery to subscribed recipients. Cover debounce behaviour and message content.

B. Web Frontend

Describe the React-based dashboard hosted on AWS Amplify: live tiles, historical charts, sensor status indicators, and authentication via AWS Cognito. Include a screenshot or mock-up (see template: figure*).

VII. Evaluation

A. Testability

Discuss per-service testability: Lambda unit tests (SAM CLI), local emulation of DynamoDB / S3 / API Gateway, limitations for IoT Core (Device Advisor only), and the role of LocalStack for higher-level integration tests.

B. Cost Estimation

Present the monthly operating-cost estimate per device under realistic assumptions (users, active hours, message volume). Compare Free Tier and Post Free Tier costs across all involved AWS services. Template available at end of document: tab:aws-costs.

VIII. Summary and Future Work

Recap the contribution of the report: a working, MCU-based IoT architecture for environmental monitoring with secure ingestion, dual-path persistence, and a user-facing dashboard. Outline next steps: extended sensor coverage, predictive analytics on historical data, multi-tenant support, and hardening for production deployment.